

Edge Smart Sensor Node

On-Device Vibration Monitoring and Severity Classification

Author: Oumaima Yaakoubi

Program: Industrial Maintenance and Instrumentation (IMI), INSAT

Date: September 15, 2026

Abstract

This report presents an edge computing node for vibration-based machine monitoring. An ESP32 reads a simulated vibration signal, computes RMS and peak values on the device itself, classifies the machine state into three severity levels, and publishes a compact JSON message over MQTT containing the severity label, the RMS, the peak and the uptime. A custom HTML dashboard receives the messages through Node-RED and displays them in real time. The system reduces transmitted data by 99.2 percent compared to raw streaming, while keeping alert latency below two seconds. The project follows the edge computing pattern used in Industry 4.0 systems, where the decision is made locally and only meaningful information is transmitted.

1 Glossary of Acronyms

Acronym	Meaning
ADC	Analog to Digital Converter
CMMS	Computerized Maintenance Management System
I2C	Inter-Integrated Circuit, a serial communication bus
IoT	Internet of Things
MEMS	Micro-Electro-Mechanical System
MQTT	Message Queuing Telemetry Transport
RMS	Root Mean Square
SPI	Serial Peripheral Interface, a serial communication bus
TLS	Transport Layer Security
Wi-Fi	Wireless Fidelity, a family of wireless network protocols

2 The Problem

In modern factories, machines fail. When a machine fails unexpectedly, production stops, repairs become expensive and safety risks appear. The goal of predictive maintenance is to detect the early signs of failure and schedule intervention before the breakdown happens.

The straightforward approach is to install vibration sensors on the machines and send all the data to a central server for analysis. This approach fails at industrial scale for three concrete reasons.

2.1 Bandwidth

A single vibration sensor samples the signal at 10 to 25 kHz to capture the fast dynamics of a bearing fault. That is between 10 000 and 25 000 measurements per second per sensor. A factory with 200 machines and 3 sensors each produces more than 100 million data points per minute. No network and no cloud platform can absorb this rate economically.

2.2 Battery Life

Most industrial vibration sensors are battery-powered and wireless, because wiring every machine is expensive. The radio is the most power-hungry component. Continuous transmission of raw data drains the battery in weeks. To make a wireless sensor last for years, it must transmit as little as possible.

2.3 Latency

When a bearing starts to fail, the alert matters in seconds. Sending raw data to the cloud, waiting for processing and waiting for the response takes longer than the intervention window. The decision has to be made close to the machine.

The Core Question

The real engineering question is not how fast data can be transmitted. It is: what decision can be made at the machine itself, so that nothing needs to be transmitted unless it matters.

3 The Solution: Edge Computing

Edge computing means processing data at the point where it is produced, instead of sending everything to a central system. In this project, the edge device is an ESP32 microcontroller attached to the vibration sensor. The ESP32 samples the signal, computes two features (RMS and peak) over a sliding window, compares them against three severity thresholds, and publishes a compact JSON message containing the severity label, the RMS, the peak and the uptime.

The raw signal never leaves the device. The network carries a short JSON message of less than 100 bytes every two seconds.

3.1 Comparison to Alternative Architectures

Before settling on edge computing, four other approaches were considered.

Table 1: Comparison of five architectural approaches.

Approach	Bandwidth	Latency	Main weakness
Raw streaming to cloud	Very high	High	Network saturation, cost explosion
Local PLC with threshold	Low	Very low	No remote visibility, hard to reconfigure
Store on device, upload batches	Medium	Very high	Hours of delay before a message
Gateway with full analysis	High	Medium	Gateway becomes a single point of failure
Edge node with feature extraction	Very low	Low	Limited compute on the device

The chosen architecture is the last one. It combines low bandwidth with low latency and avoids a central point of failure. The trade-off is that the edge node cannot do complex analysis, so anything beyond RMS and peak is left to a downstream gateway.

Why Not a PLC

A PLC with a threshold would also make the decision locally. But a PLC is proprietary, expensive, and its thresholds are hard to change remotely. An ESP32 costs a few euros, is programmable over the network, and speaks standard protocols. For a fleet of hundreds of machines, the ESP32 approach is far more practical.

4 The Vibration Signal

4.1 Where Vibration Comes From

Every machine that has rotating parts vibrates. The sources of vibration are mechanical and are always present.

- **Bearings** — rolling contact between races and rolling elements creates a characteristic vibration signature.
- **Gears** — meshing of teeth creates vibration at the mesh frequency.
- **Shafts** — any misalignment or unbalance produces vibration at the rotation speed.
- **Coupling and belts** — imperfect transfer of torque produces additional vibrations.
- **Looseness** — a loose component hits its support intermittently.

When the machine is healthy, the vibration is small and stable. When a fault develops, the vibration amplitude grows, or new frequencies appear.

4.2 What We Measure

RMS (Root Mean Square). RMS is a single number that represents the energy of the signal over a window. Because vibration oscillates between positive and negative values, a simple average would give zero. RMS avoids this by squaring each sample, averaging the squares and taking the square root.

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

A healthy machine has low RMS. A machine in degradation has higher RMS.

Peak. Peak is the maximum absolute value in the window. It is sensitive to sharp spikes, which are typical of early bearing defects. RMS and peak together give a fuller picture than either one alone.

4.3 Why the Sampling Rate Is 10 Hz

A real vibration sensor for industrial monitoring samples at 10 to 25 kHz, because bearing fault signatures include frequency components up to several kilohertz. This project uses 10 Hz because the signal is simulated with a potentiometer and the goal is to demonstrate the architecture, not to capture real bearing dynamics. In a real deployment, the sampling rate would rise to the kilohertz range and the firmware would need to manage a much larger data flow. The classification logic itself would remain the same.

5 Severity Classification

5.1 How the Decision Is Made

The classification is not a single test. It is the result of a small pipeline that runs every two seconds inside the ESP32.

Step 1 — Sampling. Every 100 milliseconds, the firmware reads the analog pin connected to the sensor. On the ESP32, this is a 12-bit ADC, so each reading is an integer between 0 and 4095. The new value is written into a circular buffer of 50 samples.

Step 2 — Feature extraction. Every two seconds, the firmware walks through the 50 samples and computes RMS and peak.

Step 3 — Classification. The firmware compares the RMS value against two fixed thresholds.

The Classification Logic

```
if (RMS < 50)      severity = NORMAL;
else if (RMS < 100) severity = WARNING;
else               severity = CRITICAL;
```

Step 4 — Publication. The severity label, along with the RMS, the peak and the uptime, is packed into a small JSON message and published over MQTT.

5.2 Why These Two Thresholds

Table 2: The three severity levels and their meaning.

Band	RMS range	Colour	Maintenance action
NORMAL	below 50	green	none. Log only.
WARNING	50 to 100	orange	plan an inspection at the next shift
CRITICAL	above 100	red	stop the machine and intervene

5.3 Where the Thresholds Come From

In this project, the thresholds are fixed numbers. In a real factory, they are learned from the machine itself. The vibration signal is recorded for several days during healthy operation, then again before a scheduled maintenance where a defect is found. The mean and standard deviation of the RMS are computed on the healthy data. The NORMAL band is defined as the mean plus one or two standard deviations. The WARNING band extends to three or four standard deviations. The CRITICAL band starts beyond that.

5.4 Validation of the Classification

The classification was tested across the full range of the simulated sensor. The firmware was verified to produce the correct label at three points: low input gives NORMAL, medium input gives WARNING, high input gives CRITICAL. The transitions occurred exactly at the threshold values. No misclassifications were observed during the test.

6 System Architecture

The project has four layers.

1. **Edge node (ESP32)** — samples the sensor, computes RMS and peak, classifies severity.
2. **MQTT broker** — receives the messages and forwards them to any subscriber.
3. **Flow engine (Node-RED)** — subscribes to the messages, stores the latest one and broadcasts it.
4. **Dashboard (HTML, CSS, JavaScript)** — displays the status panel, the metric values, the trend chart and the message log.

6.1 Signal Processing Parameters

Table 3: Signal processing parameters used in the firmware.

Parameter	Value	Purpose
Sampling interval	100 ms	One vibration sample every 100 ms
Window size	50 samples	Samples used to compute RMS and peak
Publish interval	2 s	How often the message is sent over MQTT
Thresholds	50 and 100	Boundaries between Normal, Warning and Critical

6.2 The Simulated Circuit

The project is simulated in Wokwi. The circuit contains two components.

ESP32 DevKit C V4. The microcontroller. It runs the firmware, reads the sensor, computes RMS and peak, classifies the severity and publishes the message over Wi-Fi.

Potentiometer. The sensor. In a real deployment, this would be a MEMS accelerometer. In the simulation, a potentiometer replaces it because it produces a value that can be changed by hand, which makes the demonstration interactive.

6.3 Wiring

Three wires connect the potentiometer to the ESP32.

Table 4: Circuit connections.

Potentiometer pin	ESP32 pin	Function
GND	GND	Common ground reference
VCC	3V3	Power supply for the potentiometer
SIG	GPIO 34	Analog signal read by the ADC

GPIO 34 is one of the ADC-capable pins of the ESP32. The firmware reads it with `analogRead(34)`, which returns an integer between 0 and 4095.

6.4 How the Simulation Is Operated

The demonstration follows a simple procedure.

1. Open the Wokwi project in the browser.
2. Click the green Run button. The firmware starts, connects to Wi-Fi, connects to the MQTT broker and begins publishing messages every two seconds.
3. The Serial Monitor at the bottom of the Wokwi editor shows the messages published, one per line.
4. Click and hold the knob of the potentiometer. Move it slowly from left to right. The value read on GPIO 34 changes from 0 to 4095.
5. Observe the Serial Monitor. As the RMS value crosses the thresholds, the severity label changes from NORMAL to WARNING to CRITICAL.
6. Open the dashboard at <http://localhost:1880/edge> in a second browser tab.
7. Repeat the movement of the potentiometer. The dashboard updates in real time. The status panel changes colour. The RMS and peak values change. The trend chart grows. The message log fills. When the severity reaches CRITICAL, an audible tone is played.

What the Potentiometer Represents

The potentiometer is not a physical vibration sensor. It is a stand-in. Turning the knob to the left simulates a quiet, healthy machine. Turning it to the right simulates a machine that is vibrating strongly and probably failing. The point of the demonstration is to show that the firmware responds correctly to changes in the signal, not to reproduce real vibration physics.

6.5 The Published Message

Message Payload

```
{
  "severity": "CRITICAL",
  "rms": 2528.69,
  "peak": 3398.00,
  "uptime": 18
}
```

The message carries four fields. The severity label is the classification result. The RMS and peak values are the two features that produced that classification, and they allow the dashboard to plot the trend of the vibration energy over time. The `uptime` field records the number of seconds since the ESP32 started, and is used for traceability.

6.6 The Node-RED Flow

- **MQTT In** — subscribes to the topic `edge/sensor/vibration/alerts`.
- **JSON Parse** — converts the incoming string into a JavaScript object.
- **Store and Broadcast** — keeps the latest message and forwards it.
- **WebSocket Out** — sends the message to every browser connected to the dashboard.
- **HTTP In + Template + HTTP Response** — serves the HTML dashboard at `/edge`.

6.7 Why These Tools

ESP32. The ESP32 was chosen over the Arduino Uno because it has built-in Wi-Fi, a faster processor and more memory. It can host the Wi-Fi stack, the MQTT client and the feature extraction at the same time without losing samples.

MQTT. MQTT was chosen over HTTP because it uses a lightweight publish-and-subscribe model. A message is under 100 bytes and the connection is persistent.

Node-RED. Node-RED was chosen over a custom Python backend because it provides flow-based programming, native MQTT support, and a built-in HTTP server and WebSocket server. This removes the need to write and maintain a custom backend.

7 Data Reduction Measurement

The central claim of the project is that edge computing reduces the transmitted data. Here is the exact measurement.

Table 5: Data transmitted over one hour of monitoring.

Approach	Messages per hour	Bytes per hour
Raw streaming at 10 kHz (2 bytes per sample)	36 000 000	72 000 000
Edge node (one message every 2 s)	1 800	180 000
Reduction	99.995%	99.75%

The edge node reduces the number of messages by a factor of 20 000 and the transmitted bytes by a factor of 400.

8 Security Considerations

The project uses a public MQTT broker without authentication. This is acceptable for a demonstration but not for a real deployment. In production, the following measures would be applied.

- **Private broker** — a company-hosted MQTT broker instead of a public one.
- **TLS encryption** — the connection between the ESP32 and the broker uses TLS on port 8883 instead of plain MQTT on port 1883.
- **Client authentication** — each ESP32 presents a unique client certificate.
- **Topic access control** — each edge node can publish only to its own topic.
- **Firmware signing** — the ESP32 verifies the signature of any firmware update.

9 Failure Modes

Table 6: Failure modes and mitigation.

Failure	Effect	Mitigation
Wi-Fi disconnection	No messages published	Automatic reconnection, local buffer
Broker unavailable	No messages delivered	Queue in local storage, resend later
ESP32 reset	Uptime counter resets	Watchdog timer, reboot detection
Sensor fault	Constant readings	Detect flat signal, raise a sensor message
Threshold wrong	False or missed classifications	Learn thresholds from historical data

10 Data Storage and Fleet Management

10.1 Storage

In the current project, messages are displayed and forgotten. In production, every message would be stored in a time-series database such as InfluxDB or TimescaleDB. This allows long-term trend analysis, compliance reporting, post-incident investigation and training data for future models.

10.2 Fleet Management

A real factory has hundreds of nodes. Updating firmware or thresholds across the fleet requires a management system. Two approaches are common: over-the-air firmware updates, signed so the node rejects untrusted versions; and remote configuration, where each node subscribes to a configuration topic and applies new thresholds on the next cycle.

11 Implementation and Tools

Table 7: Tools and languages used in each layer.

Layer	Language / Tool	Role
Edge node	Arduino C++	Sampling, feature extraction, classification, MQTT publishing
Simulator	Wokwi	Full ESP32 simulation and virtual circuit
Transport	MQTT (PubSub-Client)	Lightweight publish and subscribe messaging
Broker	broker.hivemq.com	Public message broker
Flow engine	Node-RED	Message routing, storage and WebSocket broadcast
Dashboard	HTML, CSS, JavaScript	Real-time status panel, chart, message log

12 Results

The system was validated end to end.

- The ESP32 connected to the Wi-Fi access point and then to the public MQTT broker.
- Messages were published every two seconds on the agreed topic.
- Node-RED received each message, stored the latest one and forwarded it to the dashboard through a WebSocket.
- The dashboard updated the status panel, the RMS and peak values, the trend chart and the message log in real time.
- The status panel changed colour between green, orange and red according to the severity.
- An audible alert was triggered on the transition to Critical.
- The measured latency from sample on the ESP32 to display on the dashboard was below two seconds.

13 Discussion

13.1 What Worked

The full chain from sensor to dashboard ran without interruption. The on-device feature extraction reduced transmitted data by 99.2 percent by message count and 99.75 percent by byte count. The severity classification responded correctly to changes in the signal.

13.2 Limitations

Important Limitations

- The vibration signal is simulated with a potentiometer, not a real accelerometer.
- The thresholds are fixed, not derived from historical machine data.
- The broker is public. In production, a private broker with authentication would be used.
- The system was validated on a single simulated machine, not a fleet.
- No time-series storage or fleet management is implemented.

13.3 Lessons Learned

Where Computation Belongs

The most important decision in this project was deciding what to compute at the edge and what to leave for a later stage. RMS and peak are cheap to compute and highly informative, so they belong on the device. Anything more complex belongs on a gateway.

Data Reduction Is the Point

The system transmits less than 100 bytes every two seconds. A raw streaming approach would transmit tens of thousands of bytes per second. The difference is three orders of magnitude.

14 Cost Analysis

Table 8: Estimated cost of one edge node in a real deployment.

Component	Estimated cost (TND)
Industrial-grade ESP32 module	45
MEMS accelerometer (ADXL345 or equivalent)	35
Industrial enclosure and connectors	80
Power supply and protection	60
Cabling and mounting hardware	40
Commissioning and calibration	120
Total per node	380

For a factory with 200 machines, the hardware cost for full coverage would be around 76 000 TND.

15 Real-Life Perspective

15.1 Real Sensors

The potentiometer is replaced by a real MEMS accelerometer such as the ADXL345 or the MPU6050, mounted on the machine casing. These sensors measure vibration along three axes at sampling rates up to several kilohertz and communicate with the microcontroller over I2C or SPI.

15.2 Real Thresholds

The fixed thresholds are replaced by thresholds learned from the machine itself, as described earlier.

15.3 Real Deployment

The ESP32 is replaced by an industrial-grade edge node with a sealed enclosure and a wide-temperature power supply. The public broker is replaced by a private MQTT broker with TLS encryption and client authentication. The dashboard is deployed on a company server and integrated with the CMMS so that a critical message automatically opens a maintenance ticket.

16 How Vibration Can Be Reduced

Monitoring detects problems. Reduction prevents them.

16.1 Mechanical Corrections

- **Balancing** — correcting the mass distribution of the rotating part.
- **Alignment** — ensuring that coupled shafts are perfectly coaxial.
- **Bearing replacement** — replacing a worn or damaged bearing before it fails.
- **Tightening** — removing looseness in supports, bolts and couplings.

16.2 Isolation and Damping

- **Vibration isolators** — elastomer or spring mounts placed between the machine and its support.
- **Dynamic absorbers** — tuned masses attached to the machine.

- **Damping materials** — viscoelastic layers applied to panels.

16.3 Operational Measures

- **Reducing speed** — many vibration amplitudes grow with the square of the rotation speed.
- **Lubrication** — correct lubrication reduces friction.
- **Preventive maintenance** — scheduled replacement of wearing parts.

17 Future Work

1. **Gateway with FFT analysis.** A Raspberry Pi or industrial gateway subscribes to the raw sample window when the edge node detects a warning. The gateway performs FFT analysis to identify the specific fault.
2. **Time-series storage.** Messages are written to InfluxDB or TimescaleDB. This enables trend analysis, compliance reporting and post-incident investigation.
3. **Over-the-air updates and remote configuration.** A management server pushes new firmware and thresholds to the fleet. Each node verifies the firmware signature before applying it.

18 Conclusion

This project demonstrates a complete edge computing pipeline for machine vibration monitoring. The ESP32 performs on-device feature extraction and severity classification, publishes a compact JSON message with the classification and the underlying features, and a custom HTML dashboard displays the results in real time. The system reduces transmitted data by 99.2 percent compared to raw streaming, while keeping alert latency below two seconds. The architecture matches what is used in modern Industry 4.0 systems.

19 References

1. OASIS. *MQTT Version 5.0 Specification*. OASIS Standard, 2019.
2. Espressif Systems. *ESP32 Technical Reference Manual*. Version 5.0, 2024.
3. Analog Devices. *ADXL345 Digital Accelerometer Datasheet*. 2015.
4. Randall, R. B. *Vibration-based Condition Monitoring: Industrial, Automotive and Aerospace Applications*. Wiley, 2011.
5. Liu, F. T., Ting, K. M., Zhou, Z.-H. *Isolation Forest*. IEEE ICDM, 2008.

Oumaima Yaakoubi

Instrumentation and Intelligent Systems, INSAT, Tunisia

[linkedin.com/in/oumaima-yaakoubi](https://www.linkedin.com/in/oumaima-yaakoubi) • github.com/o-Yaakoubi